

`MOUSEBUTTONUP`, OR `MOUSEBUTTONDOWN` ❸. The rest of the method is implemented as a *state machine*, a technique that I will go into more detail about in Chapter 15. The code is a little complicated, and you should feel free to study how it works, but for now note that it uses the instance variable `self.state` (over the course of multiple calls) to detect if the user has clicked on the button. The `handleEvent()` method returns `True` when the user completes a mouse click by pressing down on the button, then later releasing on the same button. In all other cases, `handleEvent()` returns `False`.

Finally, the `draw()` method uses the state of the object's instance variable `self.state` to decide which image (up or down) to draw ❹.

Main Code Using a SimpleButton

To use a `SimpleButton` in the main code, we first instantiate one from the `SimpleButton` class before the main loop starts with a line like this:

```
oButton = SimpleButton(window, (150, 30),  
                        'images/buttonUp.png',  
                        'images/buttonDown.png')
```

This line creates a `SimpleButton` object, specifying a location to draw it (as usual, the coordinates are for the top-left corner of the bounding rectangle) and providing the paths to both the up and down images of the button. In the main loop, any time any event happens we need to call the `handleEvent()` method to see if the user has clicked the button. If the user clicks the button, the program should perform some action. Also in the main loop, we need to call the `draw()` method to make the button show in the window.

We'll build a small test program, which will generate a user interface like [Figure 6-2](#), to incorporate one instance of a `SimpleButton`.